UNIVERSITÀ DI TORINO

Hybrid Workflows for Artificial Intelligence

Workflows

Prof. Iacopo Colonnelli

iacopo.colonnelli@unito.it

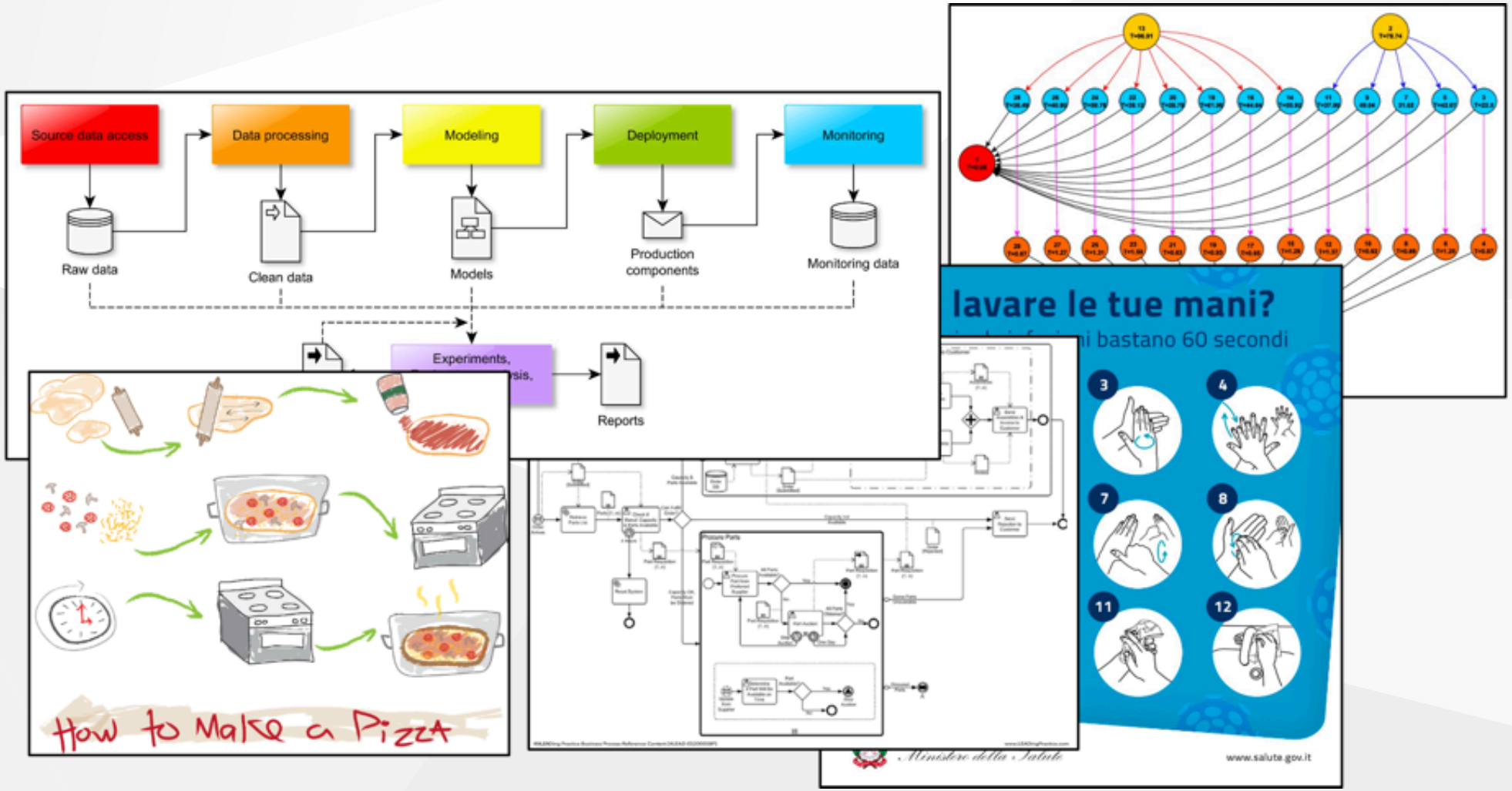
https://alpha.di.unito.it/iacopo-colonnelli

UNIVERSITÀ DI TORINO

Workflow

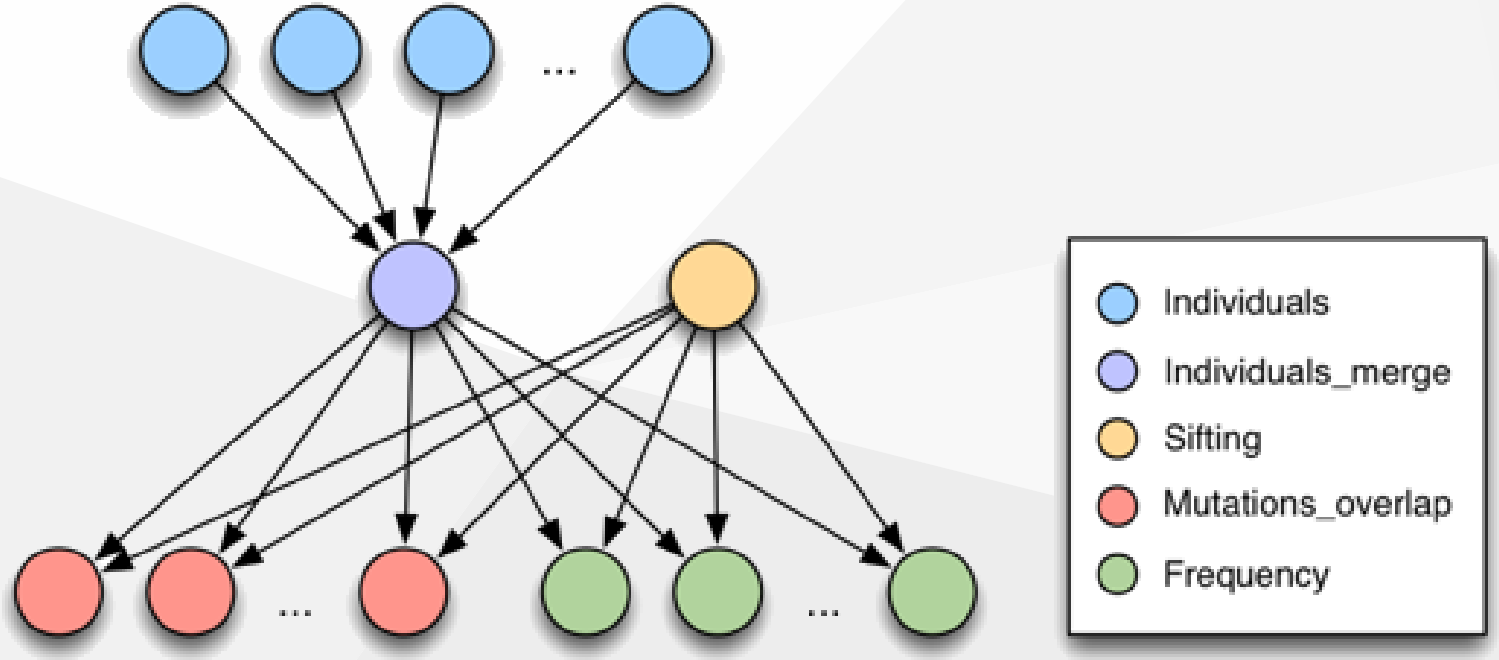
A **workflow** is an abstraction that models a complex and modular working process as a set of **steps** and their **inter-dependencies**

Workflow


Source data access
Data processing
Modeling
Deployment
Monitoring
Raw data
Clean data
Models
Production components
Monitoring data
Experiments,
Reports
How to Make a Pizza
lavare le tue mani?
bastano 60 secondi
www.salute.gov.it

Scientific workflow

In scientific research, workflows are used as a paradigm for designing concurrent distributed applications in a **portable** and **reproducible** way and executing them on **large-scale architectures**



Why workflows?

UNIVERSITÀ DI TORINO

- Workflows offer a **structured approach** to program complex applications
- A good workflow management system allows to abstract the application's business logic from the management of non-functional requirments, like **scalability**, **automation**, and **monitoring**
- Workflows provide a **powerful abstraction** to describe an application and its execution, fostering **portability** and **reproducibility**
- Almost all workflow models support **composability**: a step can contain sub-workflows. This feature fosters **maintainability** and **reusability**

UNIVERSITÀ DI TORINO

Formal definition

A workflow can be formally defined as a **directed bipartite graph** $W = (S, P, D)$, where:

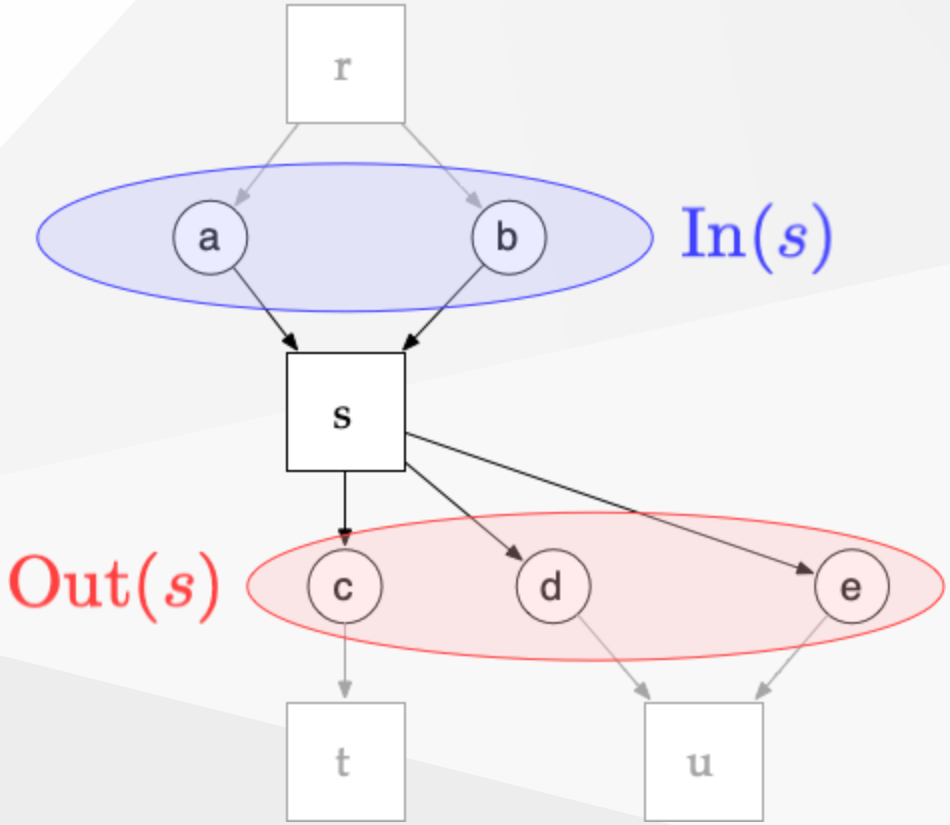
- S is the set of **steps** (the tasks to execute)
- P is the set of **ports** (the data produced and consumed by steps)
- $D \subseteq (S \times P) \cup (P \times S)$ is the set of **inter-dependencies** between steps (and ports)

A step B **directly** depends on another step A if it is possible to move from A to B by traversing one or more ports, but no other steps

Formal definition

Given a step $s \in S$, it is possible to define:

- The set of **input ports**:
  $\text{In}(s) = \{p \in P : (p, s) \in D\}$
- The set of **output ports**:
  $\text{Out}(s) = \{p \in P : (s, p) \in D\}$



UNIVERSITÀ DI TORINO

Semantics

A workflow model is composed oftwo distinct semantics:

- **Host semantics** define the subprogram in each workflow step, usually expressed in a general-purpose programming language (e.g., Java, Python, C++) or as a shell script
- **Coordination semantics** define the interactions between steps. Coordination semantics can be either interleaved with host semantics or expressed through a declarative markup syntax, an imperative Domain Specific Language (DSL), or a graph-based modelling interface

Semantics

Sometimes the two semantics can be intermingled in a single program

In this case, coordination semantics must be expressed using the same language of the host semantics

To the right, a Python pseudocode reproduces the workflow shown above

```python
@task
def r():
  return a, b

@task
def s(a, b):
  ...
  return c, d, e

@task
def t(a):
  ...

@task
def u(a, b):
  ...

def main():
  a, b = r()
  c, d, e = s(a, b)
  t(c)
  u(d, e)

if __name__ == '__main__':
  main()
```

UNIVERSITÀ DI TORINO

Semantics

Alternatively, the two semantics can be kept fully separated. In these cases, coordination semantics can be expressed through:

- A **declarative language** like JSON or YAML
- An **imperative Domain Specific Language (DSL)**, often similar to the Make syntax
- A **Graphical User Interface (GUI)** for workflow design

Worfklow life cycle

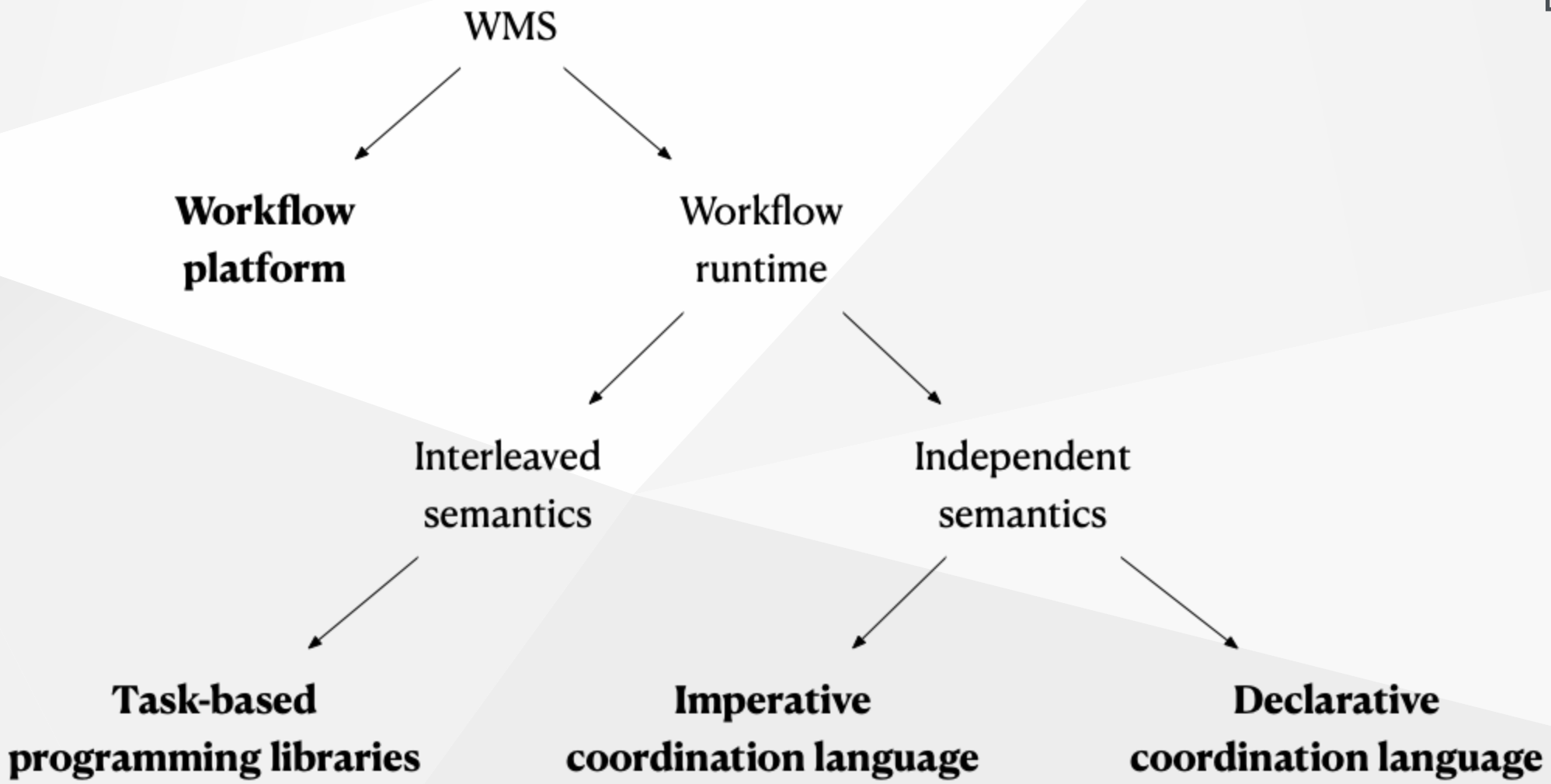
There are two main phases in the life cycle of a scientific workflow:

- During the **design phase**, a domain expert describes the different functional components of an application and their dependencies as a workflow model
- During the **runtime phase**, a WMS deploys and manages the computational units required for the workflow execution

Workflow Management Systems

Tools in charge of exposing coordination semantics to the users and orchestrating workflows are called **Workflow Management Systems (WMSs)**

Coarse-grained WMS taxonomy



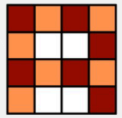
Workflow platforms

- Support **both design and runtime phases**, usually through advanced **Graphical User Interfaces (GUI)**
- Support all aspects of workflow management, e.g., **provenance collection**, **catalogs**, and **fault-tolerance**
- Usually tightly coupled with a **specific underlying architecture** (e.g., the Grid), without focusing on portability
- They are usually **complex to be installed and properly configured**, as they rely on low-level external libraries that must be independently managed (e.g., HTCondor or GAP interface)

Workflow platforms

Pegasus

Apache Taverna



ASKALON

Task-based programming libraries

- Users identify and annotate functions that can be executed as **asynchronous remote tasks**
- Synchronicity is typically implemented with the **futures paradigm**
- The workflow execution plan, typically a layered dataflow model, is **built just-in-time** by the runtime engine
- **Privilege performance** over accessibility, exposing a low-level programming model directly to the user
- Task-based programming libraries commonly offer support for a **limited set of host languages**, resulting in limited reusability and extensibility

Task-based programming libraries

APACHE STORM™

Apache Flink

RAY

Imperative coordination languages

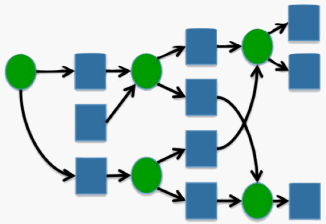
- Workflow are described using an **imperative Domain Specific Language (DSL)**, which is commonly a subset of a general-purpose programming language
- **Apache Airflow** and **Snakemake** are essentially Python scripts extended by declarative code that can be executed on distributed infrastructures
- **Makeflow** exposes a technology-neutral syntax similar to Make
- The **Nextflow** framework builds on the Unix pipe concept to expose an explicit dataflow model
- **Toil** and **DagOnStar** model workflows as pure Python scripts, through dedicated APIs

Imperative coordination languages

Apache Airflow

nextflow

Makeflow



Declarative coordination languages

UNIVERSITÀ DI TORINO

- The **Common Workflow Language (CWL)** is an open standard for describing workflow DAGs following a JSON or YAML syntax
- Other examples of workflow modelling open standards are the **Workflow Description Language (WDL)** and the **Serverless Workflow Specification**
- Declarative coordination languages are commonly **less expressive** than imperative ones, but it's easier for WMSs to apply rewriting techniques for optimization, improving **performance portability**
- Also, declarative languages are usually **product-agnostic**, improving portability and reusability. An exception is **DAX**, the Pegasus' XML-based low level representation of DAGs

Declarative coordination languages

Pegasus

COMMON
WORKFLOW
LANGUAGE

{wdl}

Serverless
Workflow
Specification

UNIVERSITÀ
DI TORINO

How to choose a WMS?

- Many existing WMSs share some salient features, offer similar functionalities, and can manage the same categories of workflows
- At the same time, they have some **distinct capabilities** that can be important for specific applications
- Currently there are **several hundreds** of workflow systems, even if not all of them are actively maintained
  - https://github.com/common-workflow-language/common-workflow-language/wiki/Existing-Workflow-systems
  - https://github.com/pditommaso/awesome-pipeline

How to choose a WMS?

Choosing the best suited WMS is a **complex task**, as the "best" WMS depends on several factors:

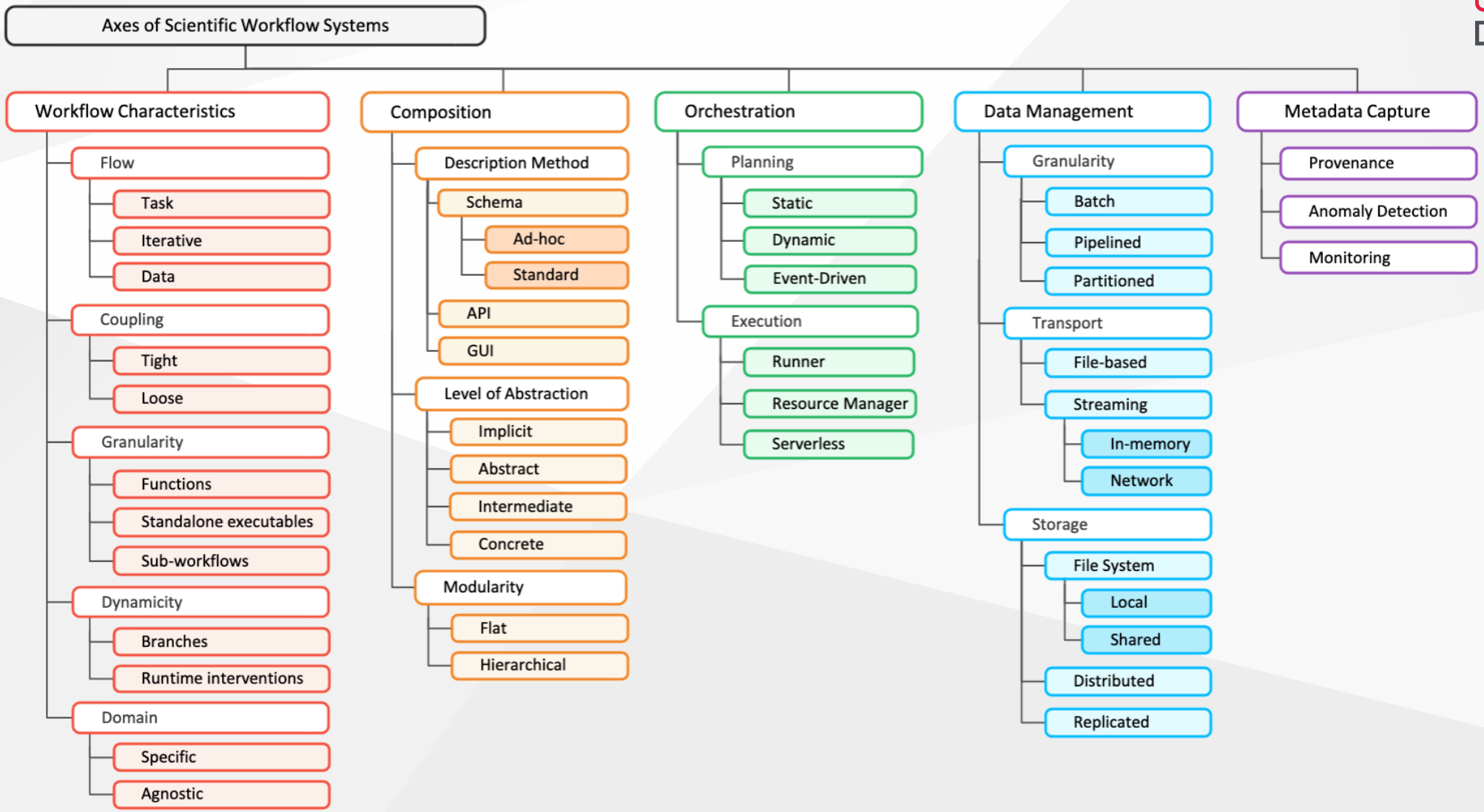
- The **scientific application** to model (high performance, high throughput, federated, ...)
- The **data lineage** of the application (batch, streaming, event-driven, ...)
- The computing and storage **resources available** at runtime (laptop, cloud VMs, HPC, ...)
- Other **non-functional factors** (reputation, adoption, expertise, community support, sustainability, ...)

WMS Terminology

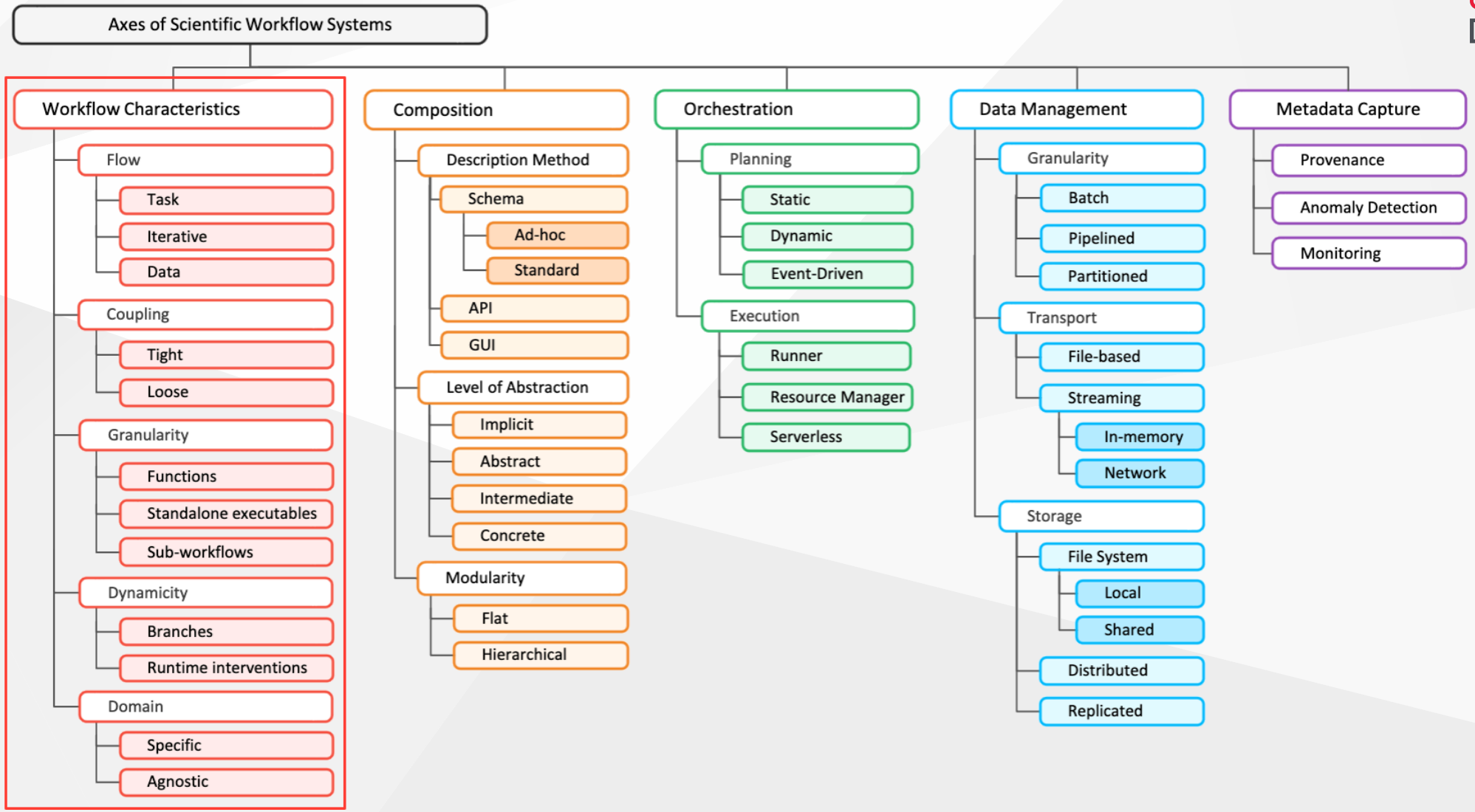
To help scientists navigate the wide range of available tools and better express their computational needs, the Workflow Community Initiative identified **five axes** to describe a workflow system:

- **Workflow characteristics** - The fundamental organizational aspects that impact how workflows operate and adapt
- **Composition** - How workflows are defined, organized, and configured by the WMS
- **Orchestration** - The implementation and execution management approaches for workflow components
- **Data management** - How data is handled throughout the workflow lifecycle
- **Metadata capture** - Additional contextual information collected during workflow execution

WMS Terminology



WMS Terminology - Workflow characteristics



WMS Terminology - Workflow characteristics

Workflow design and implementation is significantly influenced by **structural aspects** that are crucial to their **efficiency**, **scalability**, and **adaptability**. There ain't no such thing as a free lunch

Workflows can be characterised according to five defining features:

- **Flow** - how execution is driven (by tasks or data)
- **Coupling** - the nature of dependencies between these components
- **Granularity** - the level of complexity of individual components
- **Dynamicity** - the ability to modify execution paths at runtime
- **Domain** - the targeted field of application

WMS Terminology - Workflow characteristics

The **flow** of a workflow model has a direct impact on how WMSs optimise its execution:

- In **control-flow models**, the WMS maps the workflow structure on a composition of **tasks**. Each component receives inputs, processes them, generates outputs, and then terminates
- In **data-flow models**, the workflow execution is driven by the **data** flowing through the workflow components. A WMS can model these components as tasks (as before) or as **data operators** (agents, actors) that remain alive while there is data to process

Some WMSs are able to execute tasks multiple times in an **iterative** way:

- When workflow components are modelled as tasks, they are executed and terminated at each iteration
- When workflow components are modelled as darta operators, they are invoked with new parameters at each iterations

WMS Terminology - Workflow characteristics

Another defining feature of workflows is the **coupling** of the tasks that compose them, i.e., the **dependencies** and **interactions** between the different components:

- When tasks are **tightly coupled** they periodically exchange data while running. A WMS should then **co-schedule** them to run concurrently and possibly **co-locate** them on the same computing resources
- When tasks are **loosely coupled** there is no consraint on their concurrent execution, giving more flexibility to the WMS when scheduling the workflow

WMS Terminology - Workflow characteristics

The structure of workflows also differs by the **granularity** of their individual tasks:

- Some workflows can compose **function calls** to perform complex processing tasks (as in task-based programming)
- In the most common case, a workflow is a composition of **standalone executables** (scripts, programs), which aggregate multiple functions calls to perform complex computations on a set of inputs and produce a set of outputs
- Some coordination languages support expressing workflows as a hierarchical composition of **sub-workflows**, promoting reusability and modularity

WMS Terminology - Workflow characteristics

UNIVERSITÀ DI TORINO

The **dynamicity** of a workflow indicates its ability to modify its structure during execution:

- Dynamic workflows can contain **conditional branches** that are activated or not depending on the realization of a predefined condition, which may depend on input data, control signals, or resource availability
- The execution of a workflow branch may also be triggered by an **external event**, modelling time-related conditions (e.g., new data available, auto-scaling, fault tolerance, ...)
- Finally, a workflow may require **runtime intervention** (human-in-the-loop). In that case, the workflow system gives the control back to the user who started the workflow or to an automated external decision process

WMS Terminology - Workflow characteristics

Finally, it is possible to distinguish workflow systems with respect to the (scientific) **domain** they serve:

- Some systems are deeply rooted in a scientific community and thus mainly target **domain-specific** workflows
- Other workflow languages and WMSs are more **application-agnostic**

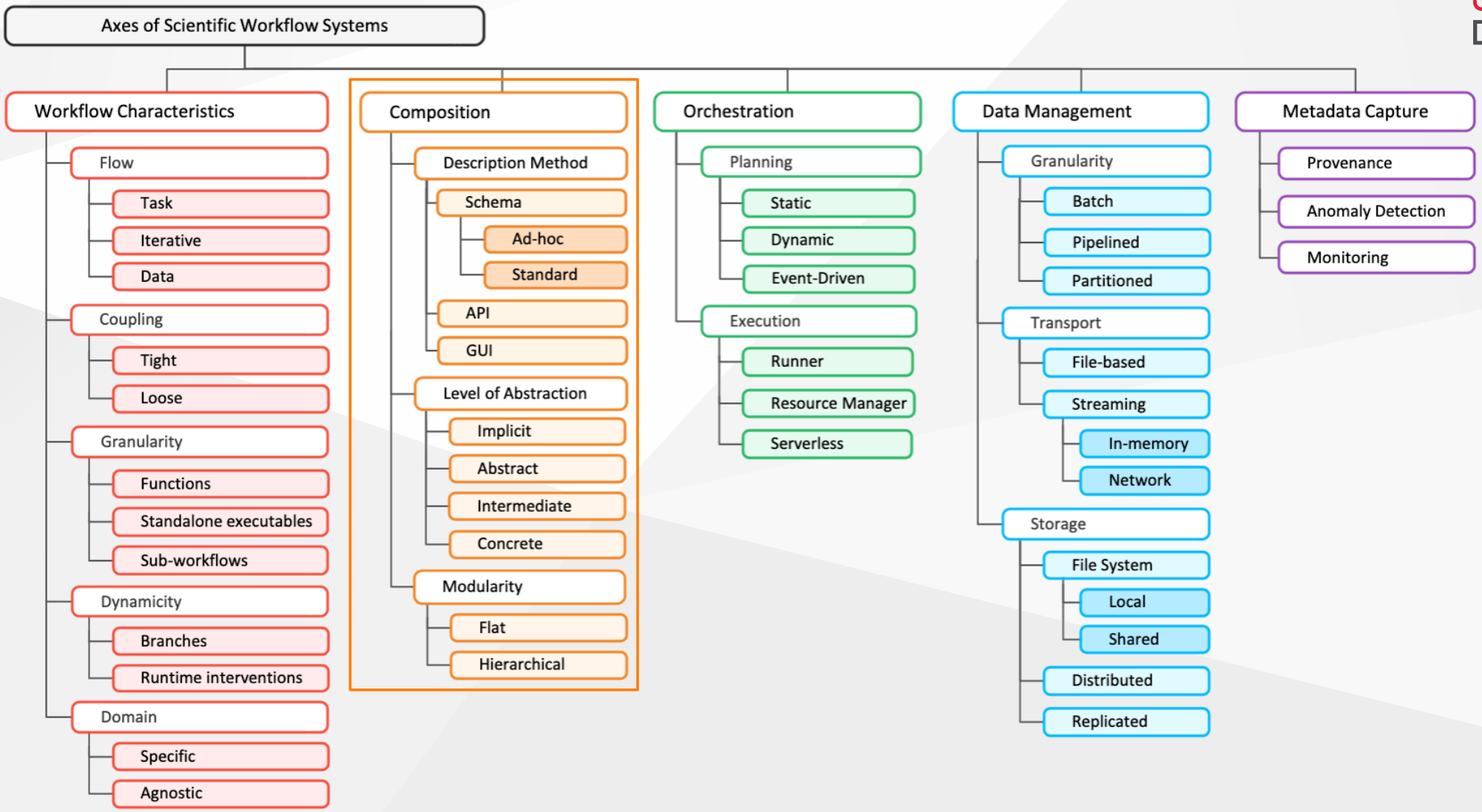
The target domain often **deeply influences** the other workflow characteristics supported by a WMS, as different domains usually deal with different workflow structures

WMS Terminology - Workflow characteristics

UNIVERSITÀ DI TORINO

Name	Flow	Granularity	Coupling	Dynamicity	Domain
AiiDA	Task - Iterative	Sub-workflows - Executables - Functions	Loose	Branches - Runtime intervention	Agnostic
AirFlow	Task	Executables	Loose	Branches	Agnostic
COMPSs	Task - Iterative	Functions - Sub-workflows - Executables	Loose	Branches	Agnostic
Dask	Data	Executables	Tight	-	Agnostic
Galaxy	Data	Executables	Loose	-	Specific
MLFlow	Task - Iterative	Executables	Loose	-	Specific
Nextflow	Data	Sub-workflows	Loose	Branches	Agnostic
Parsl	Data	Sub-workflows	Loose	Branches	Agnostic
Pegasus	Data	Sub-workflows - Executables	Loose	Branches	Agnostic
Snakemake	Task - Iterative	Sub-workflows - Executables - Functions	Loose - Tight	Branches	Agnostic
StreamFlow	Task - Data - Iterative	Sub-workflows - Executables	Loose	Branches	Agnostic

WMS Terminology - Composition



WMS Terminology - Composition

Composition refers mainly to how a workflow system allows its users to express the **description** of the different components of the workflow, the **control/data dependencies** between them, and their **configuration** and input parameters

This axis also covers the coupling between the description of the workflow itself and that of the **targeted infrastructure** (hardware and software) on which to execute the workflow

WMS Terminology - Composition

UNIVERSITÀ DI TORINO

There are three subcategories of description methods to compose a workflow:

- A workflow can be represented as a **declarative schema**, i.e., a text file with a specific format, syntax, and semantics (e.g., XML, JSON, YAML, or a DSL). A workflow schema can be **ad-hoc** (i.e., WMS-specific like Pegasus DAX) or part of a **common standard** shared by multiple WMSs (e.g., CWL, WDL, IWIR)
- Some WMSs export an **Application Programming Interface (API)** to express and execute workflows. Typically, this API builds on or extends one or more popular **programming languages** (e.g., Python, C++) or a **text templating engine** (e.g., Jinja)
- Other, more sophisticated WMSs rely on a **Graphical User Interface (GUI)**. These WMSs usually target domain experts with limited computer science expertise

WMS Terminology - Composition

UNIVERSITÀ
DI TORINO

Workflow composition can also be defined by the **level of abstraction** of the provided description:

- A **high-level abstraction** only describes the logical structure of the task graph, the data flowing through the workflow, and the amount of resources required by each component. Such description is independent of a specific workflow instance and target infrastructure
- An **intermediate-level abstraction** combines a high-level abstraction with some execution details provided by the users, balancing automation and manual fine-tuning
- A **concrete composition** provides all the parameters neede to deploy and run a workflow instance on a given infrastructure
- An **implicit composition** implies that the workflow structure is automatically inferred from the composition of the function calls made by the user (as in task programming libraries)

Some WMSs adopt a **hierarchical abstrction**, transpiling a high-level abstraction into a concrete one that can still be manipulated by the advanced users

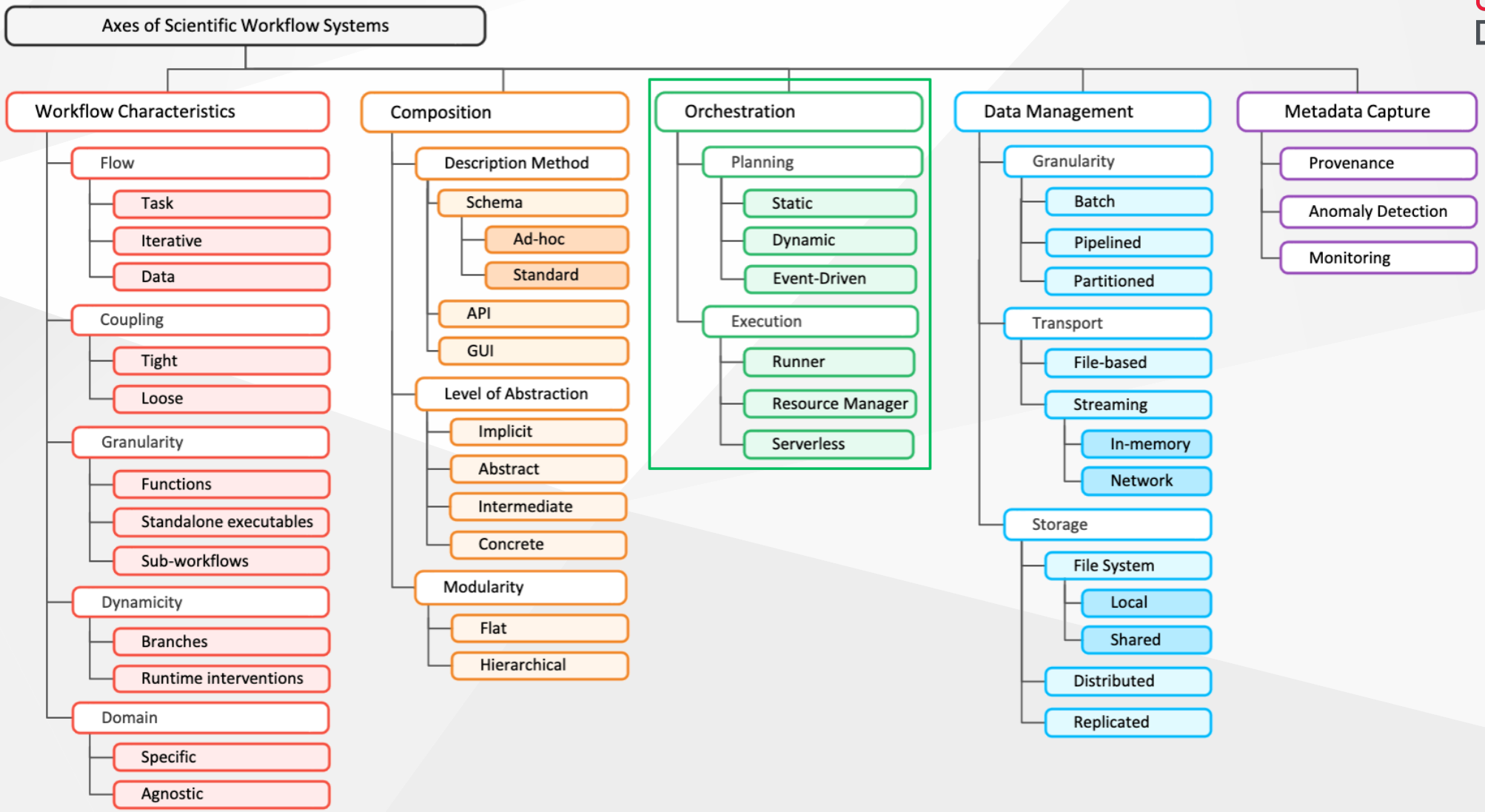
WMS Terminology - Composition

Workflow composition also affects the **modularity** of workflow representations:

- A **flat description** implies that the entire workflow should be self-contained, and that each component should be mapped onto a single task
- A **hierarchical description** enables modular and scalable workflow design by pipelining and nesting multiple, independent workflow descriptions

Modern WMSs provide features such as **parameterized workflow components** and **reusable templates** that facilitate modular workflow design while maintaining scalability

WMS Terminology - Orchestration



WMS Terminology - Orchestration

Orchestration refers to the method(s) employed by a workflow system to **deploy**, **schedule**, and **execute** the computational components of a workflow

WMSs can be classified into three broad categories related to **execution planning**:

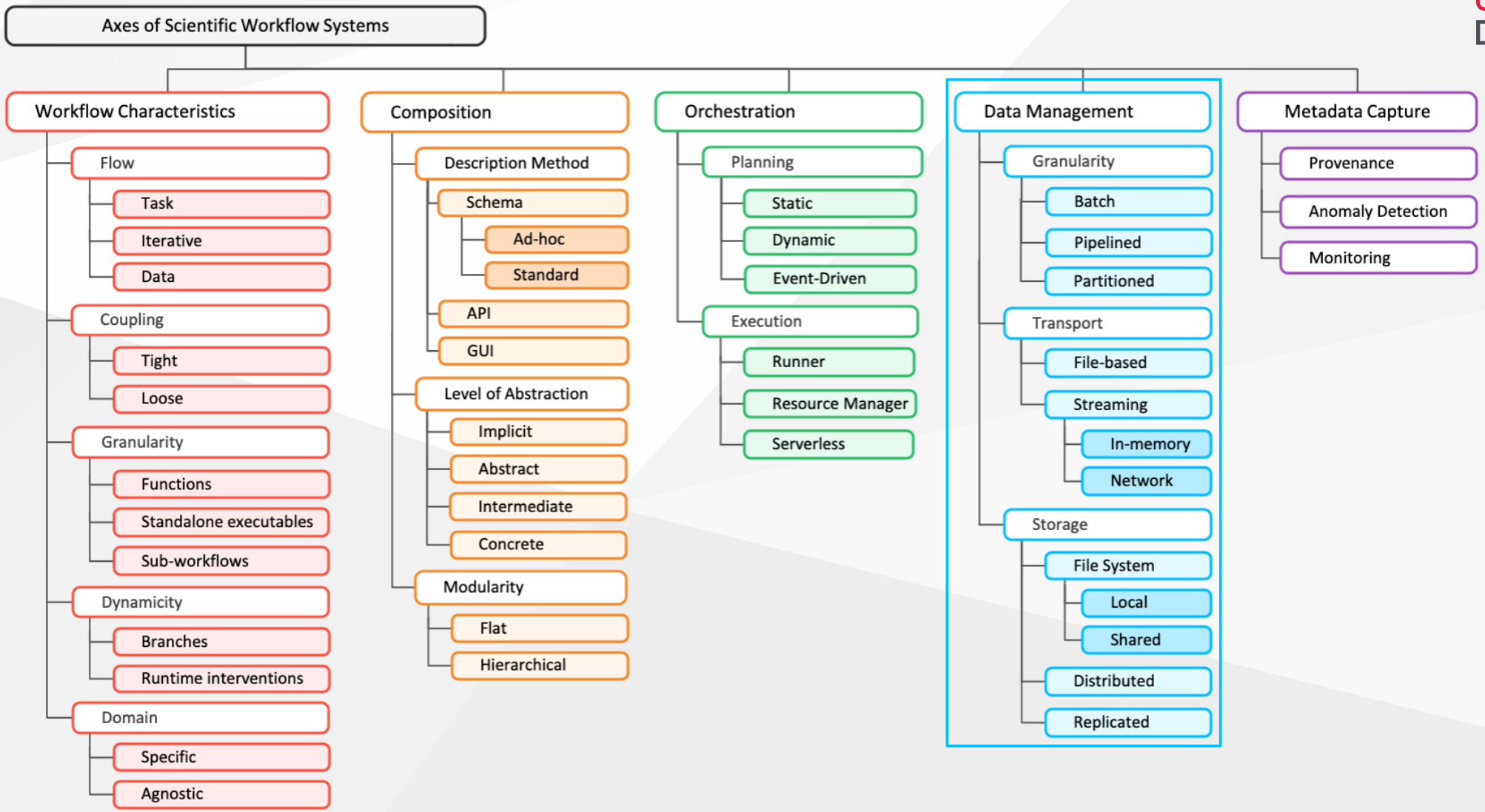
- Some systems impose a **static planning** of the workflow execution, i.e., all the decisions about when and where each task is executed must be taken before the execution starts
- Other systems can make or adapt scheduling and resource allocation decisions during the execution of the workflow, hence implementing a **dynamic planning** strategy
- Finally, some WMSs adopt an **event-driven planning**, letting the execution react to specific events and/or conditions that occur at runtime

WMS Terminology - Orchestration

WMSs also differ on how they manage the actual **task execution**:

- In the **runner** orchestration method, a WMS is fully responsible for the acquisition of computing and storage resources and the management of the individual tasks that compose a workflow
- A WMS can also delegate resource allocation and part of the management of the execution of individual tasks to an external **resource manager** (e.g., batch scheduler, container orchestrator, ...)
- The **serverless** orchestration method adopts a (mainly) cloud-based model in which the responsibility for infrastructure management, allocation scaling, and job execution is entirely delegated to an external service provider

WMS Terminology - Data management



WMS Terminology - Data management

The data management axis characterizes the way WMSs **transport**, **store**, and **manage** the data lifecycle

There are two families of data involved in a workflow execution:

- **Input/output data** respectively refer to the data needed at the beginning of the workflow and to the final outcomes of its execution. They must always live after workflow completion
- **Intermediate data** denotes every piece of data that did not exist before the beginning of the workflow and will not be used after the end of this execution. Such data can be deleted when a workflow completes, even if sometimes they are kept for provenance purposes

WMS Terminology - Data management

A first way to distinguish WMSs according to how they manage data is to consider the **granularity** at which these systems handle data management operations:

- Many WMSs consider data operations at a **batch granularity**: all the needed input data are consumed before performing computations and all the output data is produced, and made available to subsequent components in the workflow, at the end of these computations. This paradigm leads to the standard **post-hoc** execution
- Other products adopt a **pipelined granularity** in which (typically tightly-coupled) workflow components periodically produce/consume individual records during their entire lifecycle
- Finally, some libraries consider data at a **partitioned granularity**, i.e., they divide data in groups of individual records and transfer these partitions across workflow components (typically to improve performance)

WMS Terminology - Data management

A second way to differentiate WMSs is by how they **transport** data from one workflow component to another:

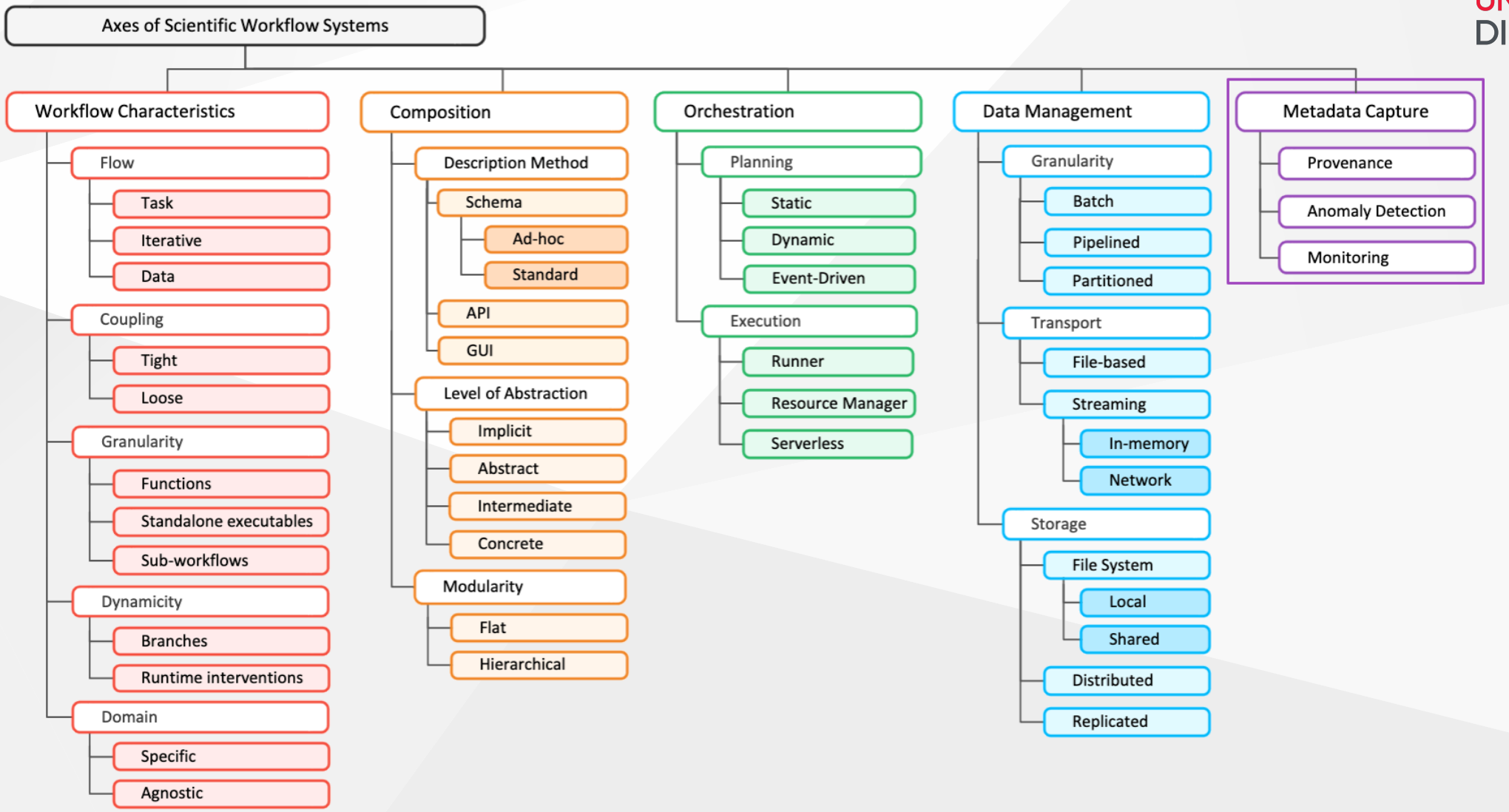
- Many WMSs rely on **file-based transport**. A workflow component that produces intermediate data writes them into a file on a storage system. Then, another workflow component consumes those intermediate data by reading from file(s)
- An alternate approach is to directly **stream** intermediate data between components:
  - When workflow components are co-located on the same compute node, the WMS can leverage on **shared memoty**
  - When components are allocated to different machines, the WMS must use the **network connection**
  - Alternatively, the WMS can demand data transfer to a **low-level communication library** (e.g., MPI, UCX, gRPC) that leverages the best suited communication strategy

WMS Terminology - Data management

WMSs that rely on storage technologies can be further divided into subclasses, depending on which kind of storage they rely on:

- When workflow components are co-located on the same compute node, the WMS can leverage on a **local file system**
- When components are allocated to different machines, it will have to rely on a **shared file system**
- When components run in distributed systems, the data-producing workflow components create additional **transfer tasks** to send data to each their its data-consuming successors
- Aternatively, the WMS can rely on **replicated storage**, which focuses on creating redundant copies of data to improve reliability, availability, and resilience

WMS Terminology - Data management



WMS Terminology - Metadata capture

The last axisrefers to the different categories of metadata, i.e., **contextual information** captured by WMSs during a workflow execution. Metadata collection serves multiple purposes:

- Enables efficient **orchestration**, **automation**, long-term **data management**, and **resource optimisation**
- Improves **troubleshooting**, **debugging**, and **responsiveness**
- Ensures **scientific integrity**, **reproducibility**, and **reliability** throughout the workflow's lifecycle

WMS Terminology - Metadata capture

A specific type of metadata is **provenance data** which can be further decomposed into two categories:

- **Prospective provenance** collects detailed information about the workflow design and structure, the configuration of the workflow system, the target computing and storage infrastructure, and other task-specific information
- **Retrospective provenance** describes what actually happened to the data processed by a workflow and captures information related to a specific workflow execution, i.e., data lineage, timestamps, and runtime details

WMS Terminology - Metadata capture

Another type of metadata captured during workflow executions is **monitoring data**, which comes from processes that oversee the workflow execution in real time

The data generated by monitoring provides critical insight into **performance**, **resource utilization**, and **potential bottlenecks**

WMSs can leverage it to dynamically **reconsider the execution plan** by modifying resource allocations or scheduling decisions, relying on some performance model (roofline, logP, ...)

The monitoring data can also be **manyally analysed** by researchers after a workflow execution to optimize the description of the workflow itself, improving its efficiency

WMS Terminology - Metadata capture

The final category on this axis is related to **anomaly detection**, i.e., the ability of a WMS to capture metadata that can be used to implement **fault tolerance mechanisms**. These mechanisms can vary in sophistication:

- Some systems just **gracefully terminate** the workflow execution and display an error message
- Others systems **complete the execution** of all active branches but log warnings about potentially incorrect data resulting from unexpected behavior
- Other systems are able **distinguish anomalies** that can be handled automatically (e.g., task retries or by an optional branch from a task-failed trigger) and anomalies that the workflow is not designed to handle and thus require **user intervention**

WMS Terminology

Name	Description	Abstraction	Modularity	Planning	Execution	Transport	Storage	Capture
AiiDA	API	Intermediate	Hierarchical	Dynamic	Runner	File-based	Shared	Anomaly Provenance
AirFlow	API	Intermediate	Flat	Static	Runner	Stream	Shared	Monitoring
COMPSs	API	Intermediate	Flat Hierarchical	Dynamic	Resource Manager Serverless	Stream File-based	Local Shared Distributed	Anomaly Monitoring Provenance
Dask	API	Concrete	Flat	Dynamic	Runner	Stream	Shared Distributed	Anomaly Monitoring Provenance
Galaxy	GUI Ad-hoc Schema	Concrete	Flat	Event-Driven	Runner	Stream	Shared	Anomaly Monitoring Provenance
MLFlow	API	Intermediate	Flat	Static	Runner	Stream File-based	Shared Distributed	Monitoring

WMS Terminology

UNIVERSITÀ DI TORINO

Name	Description	Abstraction	Modularity	Planning	Execution	Transport	Storage	Capture
NextFlow	Ad-hoc Schema	Abstract	Hierarchical	Dynamic	Runner	Stream File-based	Shared Distributed	Anomaly Monitoring Provenance
Parsl	API	Abstract	Hierarchical	Runner	Static	Stream File-based	Shared Distributed	Anomaly Monitoring
Pegasus	Ad-hoc Schema API	Abstract	Hierarchical	Static	Runner	File-based	Shared Distributed	Anomaly Monitoring Provenance
Snakemake	Ad-hoc Schema	Abstract	Flat Hierarchical	Static Dynamic Event-driven	Runner	File-based	Shared Distributed	Anomaly Monitoring Provenance
StreamFlow	Standard (CWL)	Abstract	Hierarchical	Dynamic	Runner Resource Manager	File-based	Distributed	Anomaly Provenance

How to choose a WMS?

UNIVERSITÀ DI TORINO

The WMS terminology provide a foundation for evaluating WMSs. However, the actual selection process in practice is often significantly based on **social factors**:

- Scientists typically gravitate toward systems already in use by their immediate collaborators, departmental colleagues, or disciplinary communities, creating **adoption groups** within research domains and institutions
- The perceived credibility of a workflow system is substantially enhanced when it appears in **trusted publications** or receives endorsements from **respected colleagues**
- **Institutional knowledge transfer** plays a crucial role, as existing expertise and support infrastructures significantly lower the barrier to adoption

These social dynamics create **self-reinforcing adoption patterns** that can sometimes override purely technical considerations